

Taller Script Python

Anthonny Almeida, Micaela Sagñay, Jhanavi Molina, Damián Haro

2026-07-06

1. Objetivos

- Desarrollar un script de Python con paradigma estructurado

2. Problema

Usted es el científico de datos para un sistema de lanzamiento de cohetes. Para esto, se le solicita que determine gráficamente el alcance del cohete según al menos tres ángulos de entrada, un tiempo máximo de vuelo t . Se considera que existe una fuerza de resistencia al movimiento debido al viento y representada mediante un coeficiente b . Escribir un programa que permita obtener el alcance del cohete gráficamente, mediante la resolución de ecuaciones diferenciales ordinarias. Los parámetros como Velocidad inicial V , coeficiente de resistencia B , tiempo máximo t_{max} y ángulos de lanzamiento se ingresan desde la línea de comando. Siga las instrucciones proporcionadas en la Sección 3 para completar el taller.

3. Instrucciones

1. Obtener las expresiones matemáticas para las ecuaciones diferenciales ordinarias. Vea la Sección 5.1
2. Investigar cómo funciona la librería `solve_ivp` del paquete `scipy.integrate`
3. Investigar cómo funciona la librería `argparse` para controlar el ingreso de valores y opciones desde la línea de comandos.
4. Cree una carpeta con los archivos del proyecto denominada `cohete`.
5. Dentro de la carpeta cree un archivo `main.py` con el código proporcionado en la Sección 5.2
6. Reorganizar el código proporcionado de ejemplo para conformar un proyecto de código estructurado. Las funciones estarán dentro de una carpeta `utils` en el archivo `projectile_functions.py`. El código principal se coloca en el archivo `main.py` conforme al siguiente esquema de archivos.
 - a) Con YASnippet habilitado escriba `imp` seguido de C-TAB para escribir `import`
 - b) Escriba `main` seguido de la secuencia de comandos C-TAB. Esto genera una función llamada `main`
 - c) escriba `ifm` seguido de la secuencia C-TAB para generar la condición de ejecución del programa Python como programa principal
 - d) Use `fd` seguido de C-TAB para escribir una función con *docstrings* y use `r` seguido de C-TAB para escribir `return`
 - e) Utilice `pars` C-TAB para crear un parser y `args` C-TAB para crear un argumento de entrada del programa. Importe la librería `argparse`

```

/
├── main.py
├── utils
│   ├── __init__.py
│   └── projectile_functions.py

```

4. Entregables del taller

1. Archivo .zip con el archivo `main.py` y la librería desarrollada `projectile_functions.py` y el gráfico obtenido de una carrera.
2. Completar en la sección [] de este documento una breve descripción de lo que hacen las librerías revisadas en este taller.
3. Una vez redactado el punto anterior suba el archivo .ORG y el .PDF

5. Recursos

Puede consultar más información en el siguiente vídeo y en GitHub

5.1. Desarrollo Matemático

La fuerza de resistencia del viento se puede considerar que es proporcional al cuadrado de la velocidad y en sentido opuesto a ésta. De ahí que las fuerzas actuando en el sistema quedarían determinadas por la ecuación

$$\vec{F}_{\text{net}} = \vec{F}_g + \vec{F}_f = -mg\hat{y} - b|\vec{v}|\vec{v} \quad (1)$$

La velocidad está dada por $\vec{v} = \dot{x}\hat{i} + \dot{y}\hat{j}$

En consecuencia la Fuerza neta del sistema queda:

$$\vec{F}_{\text{net}} = -mg\hat{y} - b\sqrt{\dot{x}^2 + \dot{y}^2}(\dot{x}\hat{i} + \dot{y}\hat{j}) \quad (2)$$

Que en forma vectorial se puede escribir como

$$\vec{F}_{\text{net}} = \begin{bmatrix} -b\sqrt{\dot{x}^2 + \dot{y}^2}\dot{x} \\ -mg - b\sqrt{\dot{x}^2 + \dot{y}^2}\dot{y} \end{bmatrix} \quad (3)$$

Dado que $\vec{F}_{\text{net}} = m\vec{a} = m\langle\ddot{x}, \ddot{y}\rangle$, se tiene

$$m \begin{bmatrix} \ddot{x} \\ \ddot{y} \end{bmatrix} = \begin{bmatrix} -b\sqrt{\dot{x}^2 + \dot{y}^2}\dot{x} \\ -mg - b\sqrt{\dot{x}^2 + \dot{y}^2}\dot{y} \end{bmatrix} \quad (4)$$

De donde se obtienen el par de ecuaciones diferenciales

$$\ddot{x} = -\frac{b}{m}\sqrt{\dot{x}^2 + \dot{y}^2}\dot{x} \quad (5)$$

$$\ddot{y} = -g - \frac{b}{m}\sqrt{\dot{x}^2 + \dot{y}^2}\dot{y} \quad (6)$$

Definiendo $x' = x/g$ and $y' = y/g$, se tiene entonces

$$\ddot{x}' = -\frac{bg}{m}\sqrt{\dot{x}'^2 + \dot{y}'^2}\dot{x}' \quad (7)$$

$$\ddot{y}' = -1 - \frac{bg}{m} \sqrt{\dot{x}'^2 + \dot{y}'^2} \dot{y}' \quad (8)$$

Sea $B \equiv bg/m$, entonces considerando el cambio de notación y eliminado la ' para simplificar, se propone resolver un sistema de ecuaciones de primer orden ODE siguiente:

$$\dot{x} = v_x \quad (9)$$

$$\dot{v}_x = -B\sqrt{\dot{x}^2 + \dot{y}^2} \dot{x} \quad (10)$$

$$\dot{y} = v_y \quad (11)$$

$$\dot{v}_y = -B\sqrt{\dot{x}^2 + \dot{y}^2} \dot{y} \quad (12)$$

5.2. Código Inicial Python

```
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt

def dSdt(t,S,B):
    """
    Use fd seguido de C-TAB para generar la funcion con docstrings
    Define el sistema de ecuaciones diferenciales para el movimiento del proyectil
    con resistencia al aire
    Parametros:
        t (float): Tiempo
        S (list): Estado actual [x, vx, y, vy]
        B (float): Coeficiente de resistencia del aire. Varía desde 0.0 a 1.0
    Retorna:
        list: Derivadas [dx/dt, dvx/dt, dy/dt, dvdy/dt]
    """
    x, vx, y, vy = S
    return [vx,
            -B*np.sqrt(vx**2+vy**2)*vx,
            vy,
            -1-B*np.sqrt(vx**2+vy**2)*vy]

B = 0
V = 1
t1 = 40*np.pi/180
t2 = 45*np.pi/180
t3 = 50*np.pi/180

sol1 = solve_ivp(dSdt, [0, 2],
                  y0=[0,V*np.cos(t1),0,V*np.sin(t1)],
                  t_eval=np.linspace(0,2,1000),
                  args=(B,)) # atol=1e-7, rtol=1e-4
sol2 = solve_ivp(dSdt, [0, 2],
                  y0=[0,V*np.cos(t2),0,V*np.sin(t2)],
                  t_eval=np.linspace(0,2,1000),
                  args=(B,)) #atol=1e-7, rtol=1e-4
sol3 = solve_ivp(dSdt, [0, 2],
```

```

y0=[0,V*np.cos(t3),0,V*np.sin(t3)],
t_eval=np.linspace(0,2,1000),
args=(B,)) #atol=1e-7, rtol=1e-4

plt.plot(sol1.y[0],sol1.y[2], label=r'$\theta_0=40^\circ$')
plt.plot(sol2.y[0],sol2.y[2], label=r'$\theta_0=45^\circ$')
plt.plot(sol3.y[0],sol3.y[2], label=r'$\theta_0=50^\circ$')
plt.ylim(bottom=0)
plt.legend()
plt.xlabel('$x/g$', fontsize=20)
plt.ylabel('$y/g$', fontsize=20)
plt.show()

```

6. Descripción librerías usadas

6.1. SOLVE_{IVP}

Es una función del módulo `scipy.integrate` que permite resolver numéricamente sistemas de ecuaciones diferenciales ordinarias de valor inicial, es decir sistemas donde se conoce el estado del sistema en un tiempo inicial y se busca conocer su evolución a lo largo del tiempo mediante métodos numéricos de integración. En este taller, el sistema de ecuaciones de segundo orden que describe el movimiento del cohete con resistencia del aire, donde se transforma en un sistema de 4 ecuaciones de primer orden (posición x , velocidad x , posición y , velocidad y), lo cual es necesario porque `solveivp` solo resuelve sistemas de primer orden.

Parámetros clave usados en el taller:

- `fun (dSdt)`: Función que define las derivadas del sistema, con la forma $f(t, S, *args)$, donde S es el vector de estado $[x, vx, y, vy]$.
- `tspan`: Intervalo de tiempo de la simulación, por ejemplo $[0, t_{max}]$ donde representa el tiempo inicial y final.
- `y0`: condiciones iniciales del sistema $[x0, vx0, y0, vy0]$.
- `teval`: arreglo de puntos de tiempo específicos donde se desea la solución para tener una curva suave al graficar.
- `args`: parámetros adicionales que necesita la función derivada, en este caso $(B,)$ el coeficiente de resistencia del aire.

El resultado es un objeto con atributos como `sol.t` que es el arreglo de tiempos y `sol.y` la matriz donde cada fila es una variable de estado: `sol.y[0]=x`, `sol.y[1]=vx`, `sol.y[2]=y`, `sol.y[3]=vy`, los cuales se usan luego para graficar la trayectoria del proyectil.

6.2. ARGPARSE

Es un módulo de la librería estándar de Python, diseñado para construir interfaces de línea de comandos de forma clara y estructurada, permitiendo que un programa reciba parámetros de entrada directamente al ejecutarlo en la terminal, sin necesidad de modificar el código fuente cada vez que se quiere cambiar un valor, esto es fundamental en este taller porque según el enunciado del taller, parámetros como la velocidad inicial V , el coeficiente de resistencia del aire B , el tiempo máximo de vuelo t_{max} y los ángulos de lanzamiento deben poder ingresarse desde la línea de comandos, haciendo que el script sea flexible, reutilizable y fácil de probar con distintos escenarios sin editar `main.py`. El flujo típico de uso de `argparse` es:

- Crear el parser: `parser = argparse.ArgumentParser(description="Simulación del alcance de un cohete")`, la descripción aparece automáticamente si el usuario ejecuta el programa con `-h` o `-help`.
- Agregar argumentos: con `parser.add_argument()`, especificando:
 - El nombre del argumento como por ejemplo `-velocidad` o `-v`
 - `type`: El tipo de dato esperado sea `float`, `int`, `str`, etc.
 - `default`: Un valor por defecto si el usuario no lo especifica.
 - `help`: un texto descriptivo para el usuario.
 - `nargs=-`: permite recibir múltiples valores en un mismo argumento, útil para los ángulos de lanzamiento.
- Procesar los argumentos: `args = parser.parse_args()` lee lo que el usuario escribió en la terminal.
- Usar los valores: se acceden como atributos del objeto `args`, por ejemplo `args.velocidad`, `args.B`, `args.tmax`, `args.angulos`.

Argparse también genera automáticamente mensajes de error si el usuario ingresa un tipo de dato incorrecto y un mensaje de ayuda formateado que se llama `-help`, lo cual mejora la usabilidad del programa frente a tener que editar el código fuente para cambiar los parámetros.

6.3. NUMPY

Es la librería fundamental de Python para computación científica y cálculo numérico, su estructura principal, el `ndarray` que es un arreglo n-dimensional, permite representar vectores, matrices y tensores y realizar operaciones matemáticas sobre ellos de forma vectorizada, aplicando una operación a todos los elementos del arreglo simultáneamente sin necesidad de bucles explícitos como `for`, lo cual las hace mucho más rápidas y eficientes. En este taller, `numpy` se utiliza en varios puntos clave:

- Conversión de unidades angulares: Los ángulos de lanzamiento se ingresan en grados, pero las funciones trigonométricas de Python trabajan en radianes, por lo que se convierten con la fórmula $\text{angulo}_{\text{rad}} = \text{angulo}_{\text{grados}} * \text{np.pi}/180$.
- Cálculo de componentes de velocidad inicial: Usando `np.cos(theta)` y `np.sin(theta)` se descompone la velocidad inicial `V` en sus componentes horizontal (`vx0 = V*np.cos(theta)`) y vertical (`vy0 = V*np.sin(theta)`) que son necesarias como condiciones iniciales para `solve_ivp`.
- Cálculo de la magnitud del vector velocidad: Dentro de la función `dSdt` se usa `np.sqrt(vx**2 + vy**2)` para obtener la rapidez instantánea del proyectil, un término indispensable en las ecuaciones diferenciales, ya que la fuerza de resistencia del aire es proporcional al cuadrado de la velocidad y actúa en dirección opuesta al movimiento.
- Generación de arreglos de tiempo: `np.linspace(0, tmax, 1000)` crea un arreglo de 1000 valores equiespaciados entre 0 y `tmax`, usado en el parámetro `t_eval` de `solve_ivp` para obtener una solución evaluada en puntos regulares, lo que produce una curva suave al graficar.

6.4. MATPLOTLIB

Es la librería más utilizada en Python para la creación de gráficos y visualizaciones de datos en 2D, su submódulo `matplotlib.pyplot` ofrece una interfaz similar a la de MATLAB, permitiendo crear gráficos con pocas líneas de código, en este proyecto aunque no aparece mencionada explícitamente en las instrucciones del taller como librería a investigar, sí es utilizada en el código para cumplir con el objetivo principal: obtener el alcance del cohete de forma gráfica. Funciones clave usadas en el código:

- `plt.plot(x, y, label="...")`: Dibuja la curva de la trayectoria del cohete para cada ángulo de lanzamiento simulado, usando los resultados obtenidos de `solve_ivp` (`sol.y[0]` para `x`, `sol.y[2]` para `y`).
- `plt.ylim(bottom=0)`: Fija el límite inferior del eje `y` en 0, evitando mostrar valores negativos de altura, ya que físicamente el cohete no debe descender por debajo del nivel del suelo en la gráfica.
- `plt.legend()`: Muestra un recuadro con la leyenda que identifica cada curva según su ángulo de lanzamiento.
- `plt.xlabel()` y `plt.ylabel()`: Etiquetan los ejes horizontal y vertical del gráfico, en este caso x/g y y/g , ya que las variables fueron adimensionalizadas dividiendo por la gravedad g .
- `plt.show()`: Renderiza y muestra la ventana con el gráfico final en pantalla.

Gracias a `matplotlib`, es posible comparar visualmente cómo cambia el alcance horizontal del cohete según el ángulo de lanzamiento y el coeficiente de resistencia del aire B , que es precisamente el resultado.